

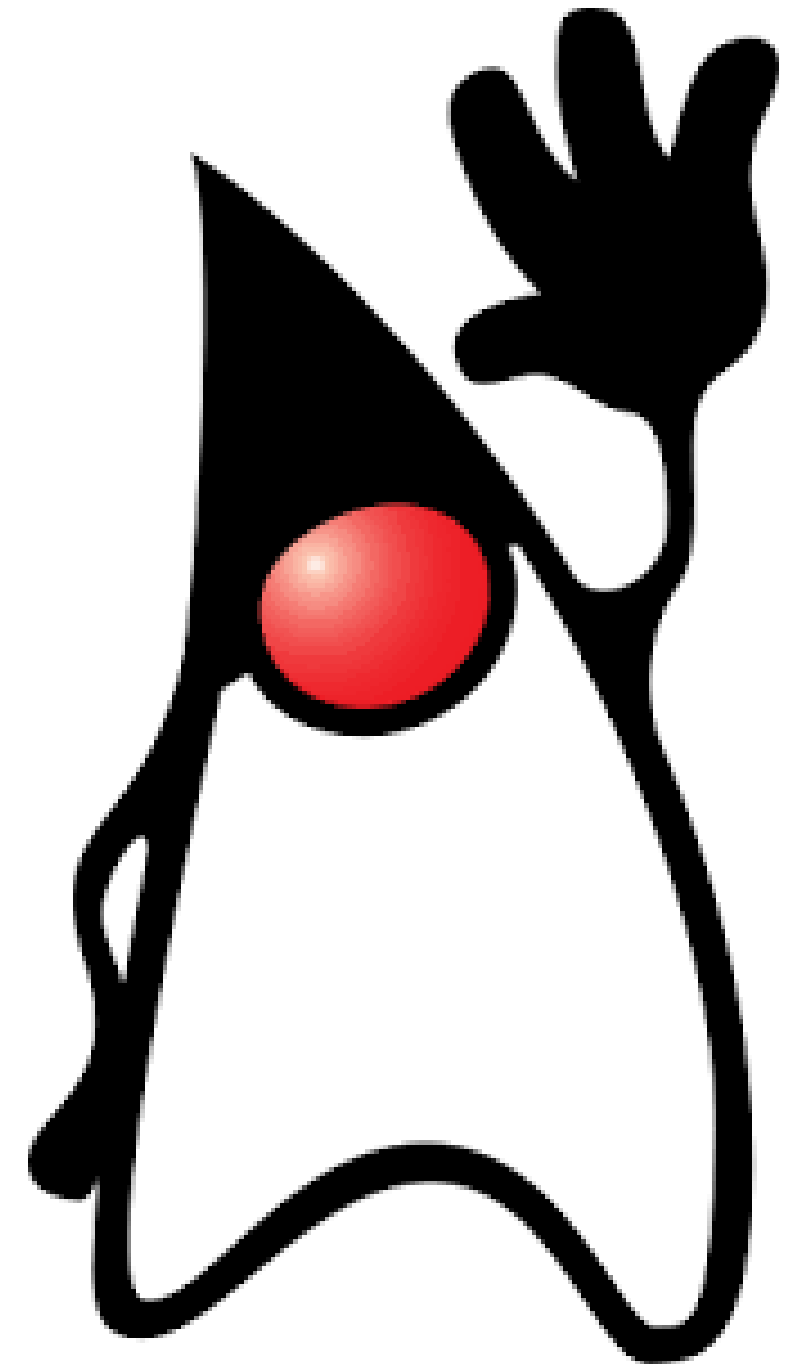
Introdução à Java

material didático



Sumário

- Sintaxe Básica;
- Operadores;
- Loops;



Sintaxe básica

| Sintaxe Básica: a estrutura mínima do código

Todo código executável em Java precisa estar dentro de uma **classe** e possuir um **ponto de entrada**, chamado **método main**.

Java

```
public *class MinhaPrimeiraClasse {  
    public static void *main(String[] args) {  
        // Exibe uma mensagem no console  
        System.out.println("Olá, Mundo!");  
    }  
}
```

| Sintaxe Básica: análise sintática

(1) public class: Define uma classe acessível. O nome da classe deve ser exatamente igual ao nome do arquivo (MinhaPrimeiraClasse.java).

```
Java
(1) public class MinhaPrimeiraClasse {
    (2) public static void main(String[] args) {
        // Exibe uma mensagem no console
        (3) System.out.println("Olá, Mundo!"); (4)
    }
}
```

(4); (Ponto e vírgula): Todo comando/instrução em Java obrigatoriamente termina com ponto e vírgula.

(2) public static void main(String[] args): É o ponto de entrada (*entry point*) do programa. É a partir deste método que a *Máquina Virtual Java* (JVM) começa a executar as instruções.

(3) System.out.println(): Função utilizada para imprimir um texto na tela e pular uma linha.

| Tipos de Dados: primitivos x não-primitivos

Existem duas categorias principais de dados: os **primitivos** e os **não-primitivos** (ou *tipo referência*). Em termos práticos, essa diferenciação implica na complexidade do dado.

Nos *tipo referência*, é possível aplicarmos **métodos** ou ainda personalizá-los; nos primitivos **os dados são brutos**, portanto eles servem apenas para armazenar valores simples.

Categoria	Tipo	Tamanho
Inteiro	byte	8 bits
Inteiro	short	16 bits
Inteiro	int	32 bits
Inteiro	long	64 bits
Ponto Flutuante	float	32 bits
Ponto Flutuante	double	64 bits
Caracter	char	16 bits
Lógico	boolean	true / false

Obs.: Java é uma linguagem **fortemente tipada**, o que significa que é necessário explicitar o tipo do dado ao declará-lo.

Operadores

| Operadores em Java

Os operadores são **símbolos especiais** que realizam operações em variáveis e valores.

A. Operadores Aritméticos

Usados para realizar cálculos matemáticos básicos.

Java

```
int a = 10;
int b = 3;

int soma = a + b;          // 13
int subtracao = a - b;     // 7
int multiplicacao = a * b; // 30
int divisao = a / b;       // 3 (Divisão inteira pois 'a' e 'b' são int)
int resto = a % b;         // 1 (Módulo / Resto da divisão)
```


| Operadores em Java

B. Operadores de Atribuição e Incremento

Atribuem e modificam valores de forma rápida.

Java

```
int x = 5;

x += 3; // Equivalente a: x = x + 3 (Resultado: 8)
x -= 2; // Equivalente a: x = x - 2 (Resultado: 6)
x++;    // Incrementa 1 unidade (Resultado: 7)
x--;    // Decrementa 1 unidade (Resultado: 6)
```



‘=’ não equivale a ‘==’

C. Operadores Relacionais (Comparação)

Comparam dois valores e retornam um valor booleano (true ou false).

Java

```
int valorA = 10;
int valorB = 20;

boolean eIgual = (valorA == valorB);    // false
boolean eDiferente = (valorA != valorB); // true
boolean eMaior = (valorA > valorB);      // false
boolean eMenorIgual = (valorA <= valorB); // true
```

| Operadores em Java

D. Operadores Lógicos

Usados para combinar múltiplas condições lógicas.

- **&& (AND / E)**: Retorna true apenas se ambas as condições forem verdadeiras.
- **|| (OR / OU)**: Retorna true se pelo menos uma das condições for verdadeira.
- **! (NOT / NÃO)**: Inverte o valor lógico (!true vira false).

Java

```
int idade = 20;  
boolean temCarteira = true;  
  
// Precisa ter idade maior/igual a 18 E ter carteira  
boolean podeDirigir = (idade >= 18) && temCarteira; // true
```

Estruturas de Repetição

ou Loops

| Estruturas de repetição:

Os Loops permitem executar um **bloco de código** repetidas vezes enquanto uma determinada condição **for verdadeira**.

Java

```
// Contar de 1 até 5
for (int i = 1; i <= 5; i++) {
    System.out.println("Contagem: " + i);
}
```

A. O Loop For: Ideal quando você sabe previamente quantas vezes o código precisa ser repetido. Ele é composto por 3 partes: **inicialização, condição e incremento**.

| Estruturas de repetição

B. O Loop for-each (Sintaxe Simplificada)

Utilizado para percorrer arrays ou listas de forma simples, sem precisar controlar o índice (i).

Java

```
String[] frutas = {"Maçã", "Banana", "Laranja"};

for (String fruta : frutas) {
    System.out.println("Fruta: " + fruta);
}
```

C. O Loop while

Executa o bloco de código enquanto a condição for verdadeira. Ideal para cenários onde você não sabe previamente quantas vezes o loop irá rodar.

Java

```
int contador = 1;

while (contador <= 3) {
    System.out.println("Executando passo " + contador);
    contador++; // Importante incrementar para evitar loop infinito!
}
```


| Estruturas de repetição

D. O Loop do-while

Garante que o bloco de código seja executado pelo menos uma vez, pois a condição é verificada apenas no final do ciclo.

Java

```
int numero = 10;

do {
    System.out.println("Este código roda ao menos uma vez!");
    numero++;
} while (numero < 5); // Condição é falsa (10 não é menor que 5), mas rodou uma vez
```

| Estruturas de repetição

Comandos de Controle de Loop

- **break:** Interrompe o loop imediatamente e sai dele.
- **continue:** Pula a iteração atual e vai direto para a próxima repetição do loop.

Java

```
for (int i = 1; i <= 10; i++) {  
    if (i == 3) {  
        continue; // Pula a exibição do número 3  
    }  
    if (i == 6) {  
        break; // Para o loop completamente quando chega no 6  
    }  
    System.out.println("Número: " + i);  
}  
// Saída: 1, 2, 4, 5
```

Atividades



| Atividades: sintaxe básica

Você é uma instituição bancária e deve registrar os dados cadastrais de um usuário. Para isso, deve coletar informações como **nome, CPF, idade, saldo em conta e data atual**; e armazená-las em suas respectivas variáveis, de acordo com os tipos de dados mais apropriados. Ao final, exiba esses dados com uma mensagem amigável.

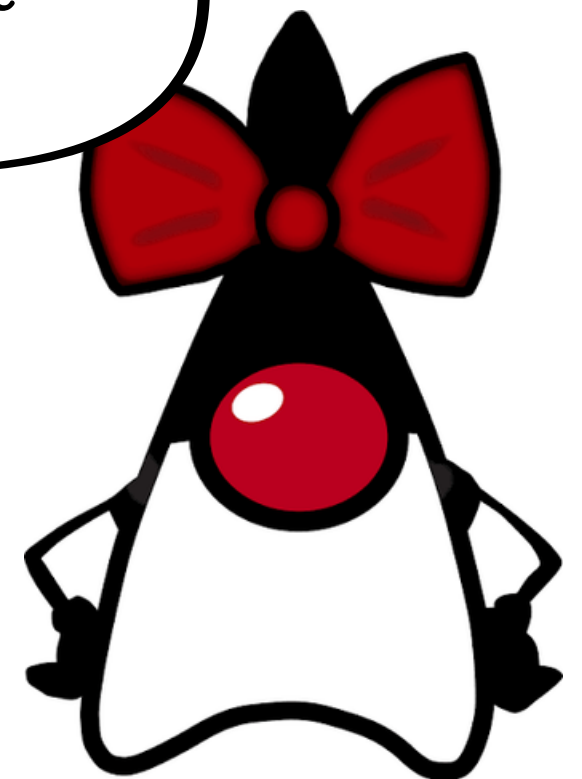
Dica: use a Classe
Scanner



| Atividades: operadores

O usuário quer consultar as operações que pode realizar em seu banco. Crie funções de **depósito** e **saque** que permitam ao usuário adicionar ou retirar dinheiro da conta. Além disso, para aperfeiçoar sua aplicação, acrescente verificações para a criação de contas (exp.: se for menor de idade, não pode criar uma conta.).

Dica: lembre do conceito
de parâmetro e
argumento



| Atividades: Loops

Crie um menu de opções que informe ao usuário as opções de navegação da sua aplicação bancária. Ele deve poder explorar a interface até que opte por encerrar sua navegação

